웹서비스설계 보고서 작성 양식

JWT 게시판 구현 보고서

01 서비스 소개와 구현 범위

학번 202422474 이름: 서준원 제출일: 2026-10-28

서비스 이름 스마트홈 플래너 커뮤니티

기술 조합과 버전 React 19.3.0 (Vite 8.3.4) / Express 5.3.0 / Node.js v22.23.2 / SQLite (sql.js 1.14.2) / jsonwebtoken 9.0.3

GitHub 저장소 URL https://github.com/se00000/web

소스 코드 기준 master 브랜치 / 테스트 기준 커밋 e89a4f0

이 서비스는 집에 스마트 센서를 설치하고 싶어 하는 사람들을 위한 커뮤니티이다. 개인적으로 IoT 에 관심이 많아서 스마트 센서에 대해 좋아하는데 설치 방법이나 기기 후기, 질문을 나누는 공간을 만들고 싶어 기획했다. 게시글은 로그인 없이 열람이 가능하고 글 작성은 로그인한 사용자만이 가능하며 수정,삭제는 작성자만이 가능하도록 하였다.

React 와 Express 를 사용하였다. 둘 다 JS 를 사용하기 때문에 같은 언어로 통일하여 사용하기 위하여 선택하였다.

제공 템플릿의 게시판 CRUD, SQLite 저장, 입력 검사, 오류 응답 형식은 그대로 사용하였다. 직접 구현은 백엔드의 JWT 발급, 검증, 작성자 권한 검사, 프론트엔드의 로그인 연결, 토큰 전송, 응답 코드별 처리, 작성자별 버튼 표시 부분을 구현하였다.

기능	완료 여부	관련 파일	비고 또는 제한
공개 목록과 상세	완료	backend/src/app.js, frontend/src/main.jsx	
로그인과 JWT 발급	완료	backend/src/auth.js frontend/src/api.js	토큰 유효 시간 90 초
인증과 내 정보 조회	완료	backend/src/auth.js	
본인 글 작성·수정·삭제	완료	backend/src/auth.js frontend/src/api.js	
만료·로그아웃·오류 표시	완료	frontend/src/api.js frontend/src/main.jsx	
DB 데이터 유지	완료	backend/src/store.js	

웹서비스설계 02

구조와 변경 내역

브라우저의 화면은 사용자의 입력을 받아 요청 모듈에 전달하고, 요청 모듈이 api 서버로 요청을 보낸다. 로그인 후 토큰을 메모리에 보관하고 요청에 토큰을 헤더로 붙인다. 토큰으로 사용자를 확인하고 권한과 입력을 검사한 뒤 SQLite 파일에 게시글을 읽고 쓴다.

로그인 요청 흐름

1. 사용자가 이메일과 비밀번호를 입력하여 로그인을 수행한다.
2. 브라우저가 api 서버로 이메일과 비밀번호를 보낸다.
3. api 서버는 입력 형식을 검사하고 확인한다.
4. 확인되면 서버가 비밀 키로 서명한 JWT 를 만들어 토큰과 사용자 정보를 응답한다.
5. 확인이 되지 않으면 오류를 응답한다.
6. 브라우저는 토큰과 사용자 정보를 메모리에 저장하고, 브라우저에 표시한다.

게시글 수정 요청 흐름

1. 작성자에게만 허용되는 수정 버튼을 클릭한다.
2. 브라우저가 메모리의 토큰을 헤더에 붙여 보낸다.
3. 서버는 인증, 글 존재 유무, 권한, 입력 순으로 확인하여 판단한다.
4. 모 통과 시 SQLite 파일의 게시글을 갱신하고 수정된 글을 응답한다.
5. 응답에 따라 토큰을 지우고 로그아웃 상태로 바꾸거나 로그인 상태를 유지한 채 권한 안내를 한다.

변경한 파일과 함수	제공 코드의 원래 역할	내가 변경한 내용	변경 이유
backend/src/auth.js issueToken	501 NOT_IMPLEMENTED 반환	HS256, sub,iss,aud,exp 를 넣어 JWT 발급	BE-01
backend/src/auth.js currentUserId	사용자 1 반환	Bearer 헤더 확인, verify 로 서명, 알고리즘, iss, aud, 만료 검증, exp 존재, sub, 실제 사용자 확인	BE-02
backend/src/auth.js assertOwner	아무 검사 없음]	요청자와 글 작성자가 다르면 403	BE-03
backend/src/app.js	"인증 TODO 미완성" 출력	서비스 이름으로 변경	미완성 문구 변경
frontend/src/api.js login	오류를 던짐	로그인 요청 후 토큰과 사용자를 session 에 저장	FE-01
frontend/src/api.js authorizationHeaders	빈 객체 반환	토큰이 있을 때만 Bearer 헤더 반환	FE-02

변경한 파일과 함수	제공 코드의 원래 역할	내가 변경한 내용	변경 이유
frontend/src/api.js request	401 처리	401 이면 clearSession()	FE-03
frontend/src/api.js canEdit	항상 true	로그인 사용자 ID 와 글 authored 비교	FE-04
frontend/src/main.jsx run	오류 메시지만 표시	404 면 상세 선택 해제 후 목록 갱신	FE-03
frontend/src/main.jsx 화면	데모 배너와 시작 코드 문구	서비스 소개, 이용 안내 배너, 비로그인 글쓰기 안내	FE-04

변경한 함수	출처
CRUD, SQLite 저장, 입력 검사, 오류 응답 형식, 계정 검증	제공 코드 유지
auth.js 세 함수, api.js 네 함수, main.jsx 수정	본인 작성. AI 에게 요구사항 설명과 문법 힌트를 받고 작성한 뒤 AI 리뷰로 오류를 수정함
jwt.verify 옵션, login()의 요청 줄	AI 가 제시한 코드를 반영함
코드 정리 (비교 연산자, 공백, 서버 시작 메시지)	AI 가 직접 수정함
프로젝트 폴더 구성, 의존성 설치, .env 생성	AI 가 수행함

로컬 실행 기록

항목	내용
운영체제와 버전	Windows 11 Home, Node.js v22.23.2, Chrome 156
새 폴더 설치	기존 작업 폴더에서 실행
환경설정	backend/.env.example 을 복사해 backend/.env 생성, JWT_SECRET 에 64 자리 16 진수 키 입력
프론트 빌드	cd frontend → npm install → npm run build → ✓ built in
백엔드 실행	cd backend → npm install → npm start → SmartHome 커뮤니티 API http://127.0.0.1:8000 출력
SQLite 파일 경로	backend/data/board.sqlite
4173 번 접속 및 API 연결	npm run preview 후 http://127.0.0.1:4173 접속, Network 에서 /api/posts 200 확인

서버 재시작 후 데이터 유지	엔드를 Ctrl+C 로 종료 후 npm start 로 재실행, 기존 글 유지 확인 (T15)
증거 위치	4 장 테스트 증거(빌드 로그, 서버 실행 화면), T01, T15

핵심 코드 설명

3 1 로그인과 JWT 발급

파일 / 함수: backend/src/auth.js (4~11 행), issueToken(12~14 행) / 호출 위치: backend/src/app.js (45~60 행)

코드 발췌

```
// backend/src/app.js
app.post("/api/auth/login", (req, res) => {
 const input = fields(req.body, { email: 254, password: 128 });
 const user = verifyCredentials(input.email, input.password);
 if (!user)
  throw new ApiError(
   401,
   "INVALID_CREDENTIALS",
   "이메일 또는 비밀번호를 확인하세요.",
  );
 const accessToken = issueToken(user);
 res.json({
  accessToken,
  expiresIn: Number(process.env.TOKEN_TTL || 90),
  user,
 });
});

// backend/src/auth.js 12~14 행
export function issueToken(user) {
 return jwt.sign({}, new_key, {algorithm: "HS256", subject: String(user.id), issuer: ISSUER, audience: AUDIENCE, expiresIn: TTL});
}
```

동작과 응답 코드 설명

사용자가 로그인 화면에서 이메일과 비밀번호를 입력하면 브라우저는 POST /api/auth/login 요청에 {“email”: ., “password”:...} JSON 형식으로 서버에 보낸다. 이 때 토큰이 아직 없으므로 Authorization 헤더를 보내지 않는다.

서버는 먼저 본문을 검사한다. 본문에는 email 과 password 두 필드만 존재하고 비어 있지 않은 문자열이어야 한다. 필드가 비어있거나 문자열이 아닌 경우 길이를 초과하였을 경우 400 BAD_INPUT 을 반환한다. 본문이 올바른 JSON 형식이 아닌 경우도 400 BAD_INPUT 을 반환한다.

입력이 올바르면 계정을 확인한다. 입력된 비밀번호를 해시한 뒤 저장된 해시와 비교 후 일치시에는 비밀번호를 제외한 {id, name, email}을 반환한다. 비밀번호가 틀리거나 존재하지 않은 계정이면 null 을 반환하며 서버는 401 INVALID_CREDENTIALS 를 응답한다.

인증에 성공하면 토큰을 발급한다. 토큰에는사용자 ID, 발급자, 사용 대상, 발급 시각, 만료 시각(90 초)가 포함된다.

로그인에 성공하면 서버는 200 OK 와 함께 {“accessToken”: “…”, “expiresIn”: 90, “user”: {“id”: 1, “name”: “Alice”, “email”: alice@class.test}}를 반환한다. 브라우저는 accessToken, user 를 메모리 변수에 저장하고 화면에 로그인 중을 표시한다. 실패 시 {“code” : … , “message” : …}형태의 JSON, 401 응답에는 WWW-Authenticate: Bearer 헤더가 함께 붙는다.

상황	HTTP	JSON code	JSON
로그인 성공	200 OK	없음	{ "accessToken": "", "expiresIn": 90, "user": { "id": 1, "name": "Alice", "email": "alice@class.test" } }
필드 누락, 문자열 X, 길이 초과 등	400 BAD REQUEST	BAD_INPUT	{ "code": "BAD_INPUT", "message": "password 입력을 확인하세요." }
비밀번호 불일치, 없는 계정	401 UNAUTHORIZED	INVALID_CREDENTIALS	{ "code": "INVALID_CREDENTIALS", "message": "이메일 또는 비밀번호를 확인하세요." }

.

3 2 브라우저 토큰 저장과 전송

파일 / 함수: frontend/src/api.js 의 session(1 행), authorizationHeaders(6~11 행), request(21~25 행), login(35~39 행) / frontend/src/main.jsx 의 로그아웃 버튼(102~103 행)

코드 발췌

```
// frontend/src/api.js 1 행

let session = { accessToken: "", user: null };


// frontend/src/api.js 6~11 행

function authorizationHeaders() {
 if(session.accessToken){
  return {Authorization: "Bearer " + session.accessToken};
 }
 return {};
```

```
}

// frontend/src/api.js 21~25 행 (request 함수 내부)
 const data = response.status === 204 ? null : await response.json();
 if (!response.ok) {
  if (response.status === 401){
   clearSession();
  }

// frontend/src/api.js 35~39 행
export async function login(email, password) {
 const data = await request("/auth/login", { method: "POST", body: JSON.stringify({ email, password }) });
 session = {accessToken: data.accessToken, user: data.user};
 return data.user;
}

// frontend/src/main.jsx 102~103 행 (로그아웃 버튼)
clearSession();
setUser(null);
```

동작과 응답 코드 설명

사용자가 로그인 버튼을 누르면 POST /api/auth/login 을 보낸다. 서버가 200 OK, {accessToken, expiresIn, user}를 반환하면 session 에 accessToken, user 를 저장한다. 이어서 화면에 로그인 중이라는 표시를 띄운다. 실패 시 400 BAD_INPUT, 401 INVALID_CREDENTIALS 의 오류가 뜨고 SESSION 에는 아무것도 저장되지 않는다.

토큰은 localStorage 나 sessionStroage, 쿠키가 아닌 메모리인 session 에 저장된다. 브라우저에 토큰을 남기지 않으므로 토큰이 노출될 위험이 줄어든다. 대신 새로고침이나 창을 닫으면 토큰이 사라져 다시 로그인을 해야한다.

토큰이 있을 때는 Authorization: Bearer <토큰>을 만들고 토큰이 없을 때 빈 객체를 반환해 헤더 자체를 보내지 않는다. 그래서 로그아웃 상태에서 Bearer undefined 같은 잘못된 값이 전송되지 않는다. 서버는 헤더의 앞 7 글자가 Bearer 인지 확인한다. 토큰이 실제로 쓰이는 요청은 POST, PUT, DELETE, GET 으로 이 중 목록과 상세 조회는 토큰 없이도 성공한다.

상황	서버 응답	JSON code	JSON
로그인	200 OK	없음	{ "accessToken": "...", "expiresIn": 90, "user": { "id": 1, "name": "Alice", "email": "alice@class.test" } }

인증 정보가 올바르지 않음	401 Unauthorized	INVALID_TOKEN	{ "code": "INVALID_TOKEN", "message": "인증 정보가 올바르지 않음" }
게시글이 존재하지 않음	404 Not Found	NOT_FOUND	{ "code": "NOT_FOUND", "message": "게시글이 없습니다." }

이 외에도 403 Forbidden, JSON code 가 다른 실패 응답들이 존재한다. 실패 응답은 모두 HTTP 상태와 code 를 오류 객체의 담는다.

로그아웃을 하면 session 을 빈 값으로 되돌리고 화면을 “로그아웃 상태”로 바꾼다. 이후 요청에는 Authorization 헤더가 붙지 않으므로, 글을 작성하려 하면 서버가 401 TOKEN_REQUIRED 를 응답한다.
로그아웃은 메모리에서 토큰을 지우지만 서버에서 토큰을 무효화하지 않는다. 따라서 로그아웃 전에 토큰은 만료 시각까지 계속 유지된다.

서버 인증과 작성자 권한

3 3 서버 JWT 검증

파일 / 함수: backend/src/auth.js 의 currentUserId(15~52 행) / 호출 위치: backend/src/app.js 의 GET /api/me(62 행), POST /api/posts(70 행), PUT /api/posts/:id(78 행), DELETE /api/posts/:id(85 행) / 401 헤더 추가: backend/src/app.js 공통 오류 처리기(102 행)

코드 발췌

// backend/src/auth.js 16~18 행: 헤더 형식 검사

const header = req.headers.authorization;

if(!header|| header.slice(0, 7) !== "Bearer ") {

throw new ApiError(401, "TOKEN_REQUIRED", "로그인 필요")

// backend/src/auth.js 27~35 행: 서명·알고리즘·iss·aud·만료 검증

try {

payload = jwt.verify(token, new_key, {algorithms: ["HS256"], issuer: ISSUER, audience: AUDIENCE});

} catch (e) {

if(e.name === "TokenExpiredError") {

throw new ApiError(401, "TOKEN_EXPIRED", "로그인 시간 만료");

}

else {

throw new ApiError(401, "INVALID_TOKEN", "인증 정보가 올바르지 않음");

}

// backend/src/auth.js 38, 42~43, 47, 51 행: 필수 클레임과 사용자 확인

if(typeof payload.exp !=="number") { // 실패 시 401 INVALID_TOKEN (39 행)

const id = Number(payload.sub);

if (!Number.isInteger(id)) { // 실패 시 401 INVALID_TOKEN (44 행)

if(!findPublic(id)) { // 실패 시 401 INVALID_TOKEN (48 행)

return id;

동작과 응답 코드 설명

요청 객체를 받아 Authorization 헤더에서 토큰을 꺼낸다. Authorization 헤더가 없거나 "Bearer "로 시작하지 않으면 401 TOKEN_REQUIRED 로 응답한다(16~19 행). "Bearer " 뒤의 토큰이 비어 있는 경우도 같은 응답을 반환한다.

Jwt.verify 는 토큰의 헤더와 payload 를 서버의 32 바이트 키로 다시 서명하여 토큰에 붙은 서명과 비교한다. JWT 의 헤더와 payload 는 암호화가 아니라 인코딩이라 누구나 내용을 고칠 수 있지만 서버의 키를 모르면 맞는 서명을 만들 수 없다. 그래서 자기 토큰의 내용을 바꿔 속일려고하여도 서명이 맞지 않아 거부된다.

HS256 으로 허용할 알고리즘을 하나로 고정하였다. 토큰 헤더의 alg 를 none 이나 다른 알고리즘으로 바꾼 토큰은 확인 전부터 거절된다.

Jwt.verify 는 exp(만료 시간)이 지나면 TokenExpiredError 를 던지고 서버는 401 TOKEN_EXPIRED 를 응답한다. 만료 외에 서명 불일치, 형식 오류, 알고리즘 오류 등은 401 INVALID_TOKEN 을 응답한다.

필수 클레임은 iss, aud, exp, sub 4 가지이며 하나라도 없거나 값이 다르면 401 INVALID_TOKEN 을 응답한다. Jwt.verify 는 iss, aud 의 값이 webservice-class, board-api 와 같은지 검사하고 exp 가 있을 경우 검사를 수행하고 없을 경우 통과시킨다. Sub 를 정수로 바꾸고 정수인지 검사한다. 마지막으로 사용자가 실제로 존재하는 사용자인지 검사한다.

모든 검사를 통과하면 사용자 ID 를 숫자로 반환한다. 각 라우트는 이 값을 받아 사용한다.

상황	HTTP	JSON code	JSON
Authorization 헤더 없음, Bearer 형식 아님, 토큰 비어 있음	401 Unauthorized	TOKEN_REQUIRED	{ "code": "TOKEN_REQUIRED", "message": "로그인 필요" }
만료 시각 경과	401 Unauthorized	TOKEN_EXPIRED	{ "code": "TOKEN_EXPIRED", "message": "로그인 시간 만료" }
sub 없음 또는 정수 아님	401 Unauthorized	INVALID_TOKEN	{ "code": "INVALID_TOKEN", "message": "인증 정보가 올바르지 않음" }
검증 성공	라우트에 따라 200, 201, 204	없음	각 라우트의 응답

3 4 작성자 권한 검사

파일 / 함수: backend/src/auth.js 의 assertOwner(53~57 행) / 호출 위치: backend/src/app.js 의 PUT /api/posts/:id(77~83 행), DELETE /api/posts/:id(84~91 행) / 작성자 결정: backend/src/app.js 의 POST /api/posts(69~76 행)

코드 발췌

```
// backend/src/auth.js 53~57 행

export function assertOwner(userId, authorId) {

 if(userId !== authorId){

  throw new ApiError(403, "FORBIDDEN", "본인이 작성한 글만 수정·삭제할 수 있음");
```

```
 }
}

// backend/src/app.js 77~83 행 (①~⑤ 주석은 설명을 위해 추가함)
app.put("/api/posts/:id", (req, res) => {
 const userId = currentUserId(req),  // ① 인증 → 401
  id = postId(req),              //  ID 형식 → 400
  p = getPost(id);               // ② 글 존재 → 404
 assertOwner(userId, p.authorId);    // ③ 권한 → 403
 res.json(store.update(id, fields(req.body, { title: 100, content: 5000 })));  // ④ 입력 → 400, ⑤ 저장
});

// backend/src/app.js 70~74 행 (글 작성 시 작성자 결정)
 const userId = currentUserId(req);
 const p = store.create(
  fields(req.body, { title: 100, content: 5000 }),
  userId,
 );
```

동작과 응답 코드 설명

userId(요청한 사람), authorId(글을 쓴 사람) 두 값을 비교한다.

발급 때 문자열로 넣은 sub 를 검증 단계에서 정수로 바꿨으므로 두 값은 모두 정수이다. 따라서 타입 변환이 없는 비교로 정확하게 비교한다. 값이 다르면 403 FORBIDDEN 을 던지고, 값이 같으면 넘어간다.

인증 - ID 형식 검사 – 글 존재 확인 – 권한 검사 – 입력 검사 – DB 변경 순으로 처리 된다.

어느 단계에서든 오류가 나면 다음 단계들은 실행되지 않는다.

401 Unauthorized 는 요청한 사람이 누구인지 확인할 수 없는 경우이다. 토큰이 없거나 변조되었거나 만료된 경우에 해당하며 currentUserId 에서 발생한다.

403 Forbidden 은 누구인지는 확인되었으나 글에 대한 권한이 없는 경우로 assertOwner 에서 발생한다.

작성자 위조를 방지하기 위해 글을 작성할 때 작성자는 클라이언트가 보낸 값이 아닌 검증된 토큰의 userId 로 서버가 정한다.

요청 본문은 title 과 content 만 허용하다. 따라서 사용자가 작성자를 바꾸려 하여도 400 BAD_INPUT 으로 거부된다.

수정할 때도 제목과 내용만이 변경되므로 작성자와 작성 시각이 그대로 유지된다.

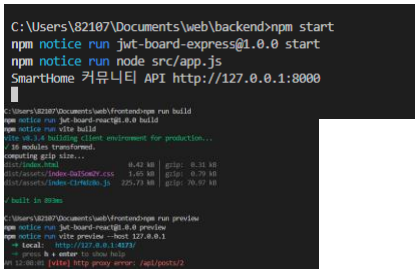
프론트엔드에서 수정, 삭제 버튼을 본인 글에만 보여준다. curl 처럼 화면을 거치지 않고 API 를 직접 호출이 가능하므로 실제 권한 판단은 반드시 서버의 assertgOwner 가 해야한다.

상황	HTTP	JSON code	JSON
자기 글 수정	200 OK	없음	{ "id": 3, "title": "수정됨", "content": "...", "authorId": 1, "authorName": "Alice", "createdAt": "...", "updatedAt": "..." }
토큰 없이 수정·삭제	401 Unauthorized	TOKEN_REQUIRED	{ "code": "TOKEN_REQUIRED", "message": "로그인 필요" }
정상 토큰으로 없는 글 수정·삭제	404 Not Found	NOT_FOUND	{ "code": "NOT_FOUND", "message": "게시글이 없습니다." }
Bob 이 Alice 글 수정·삭제	403 Forbidden	FORBIDDEN	{ "code": "FORBIDDEN", "message": "본인이 작성한 글만 수정·삭제할 수 있음" }
Alice 가 자기 글을 빈 제목으로 수정	400 Bad Request	BAD_INPUT	{ "code": "BAD_INPUT", "message": "허용된 JSON 필드만 보내세요." }

테스트 결과

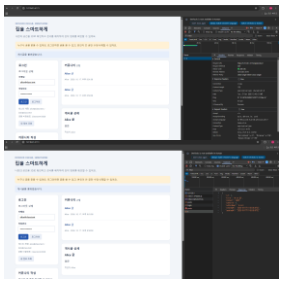
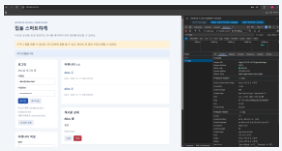
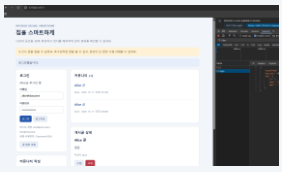
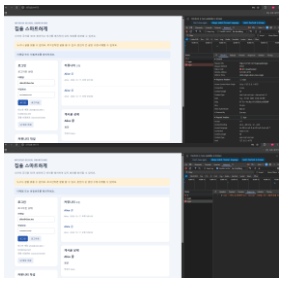
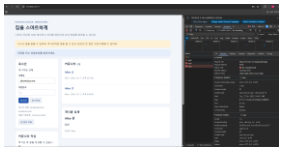
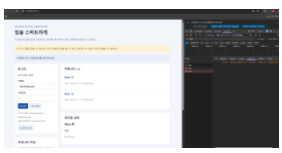
실행 환경: Windows 11 Home / Node.js v22.23.2 / Google Chrome 156 / 백엔드 cd backend → npm start (127.0.0.1:8000), 프론트 cd frontend → npm run build → npm run preview (127.0.0.1:4173)

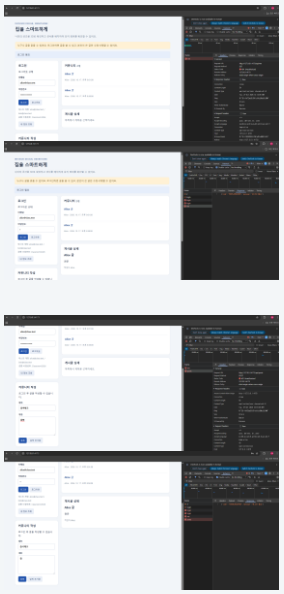
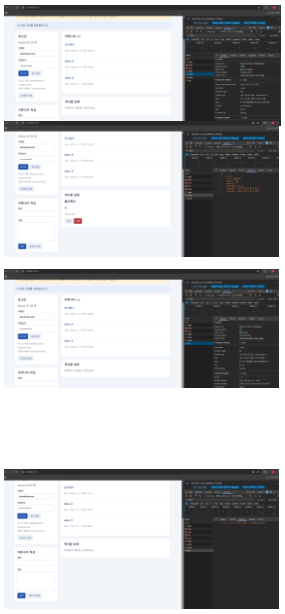
로컬 배포 확인: 제출 저장소를 새 폴더(web-clone-test)에 복제하여 처음부터 설치하였다. 백엔드는 npm ci 로 설치한 뒤 .env.example 을 복사해 .env 를 만들고 새 64 자리 16 진수 키를 입력하였으며, npm start 로 실행하였다. 프론트엔드는 npm ci 후 npm run build 로 빌드하였, npm run preview 로 실행하였다. 브라우저에서 http://127.0.0.1:4173 에 접속해 Network 탭에서 /api/posts 요청이 200 으로 응답하는 것을 확인하였다.

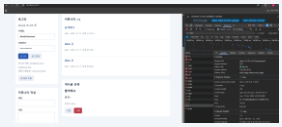
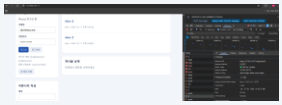
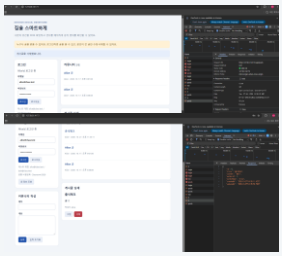
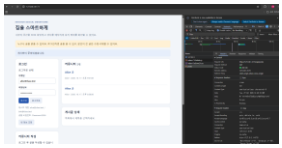
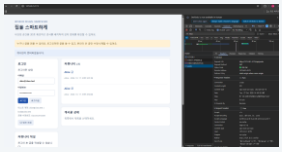


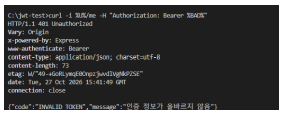
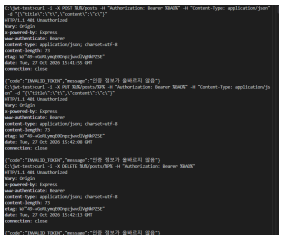
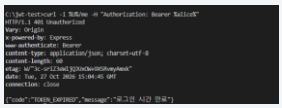
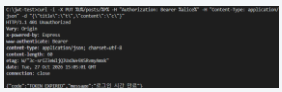
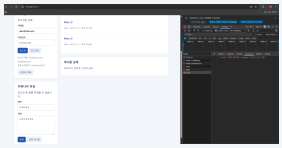
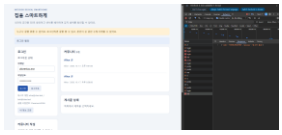
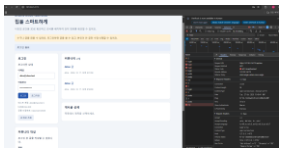
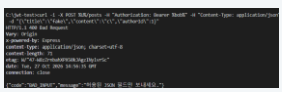
테스트 일시 및 코드 버전: 2026-10-27 / 커밋 e89a4f0

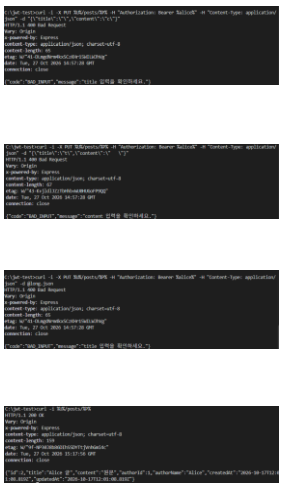
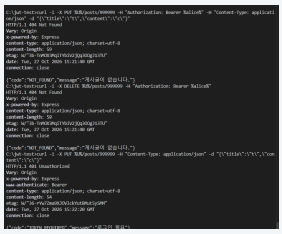
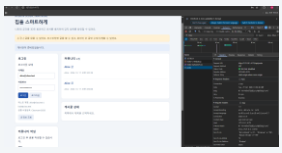
토큰 TTL: 일반 테스트 600 초 / 만료 테스트 3 초

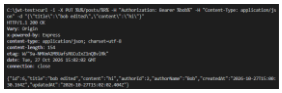
ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T01	토큰 없는 상태에서 글 조회	200/ 해당없음, 정상, 공개 조회	200 / 해당 없음	PASS	
T02	Alice 계정으로 로그인	200 / 해당없음, accessToken, user 반환, 화면에 Alice 님 로그인 중 표시	200 / 해당 없음, 화면 “Alice 님 로그인 중	PASS	 
T03	잘못된 비밀번호, 이메일로 각각 로그인	401 / INVALID_CREDENTIALS, 토큰 미발급	401 / INVALID_CREDENTIALS, 401 / INVALID_CREDENTIALS	PASS	  

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T04	토큰 없이 /me 와 글 등록 요청	각각 401 / TOKEN_REQUIRED	401 / TOKEN_REQUIRED, 401 / TOKEN_REQUIRED	PASS	
T05	Alice 토큰으로 글 작성 후 /me 조회	작성 201 / 해당없음, (authorId 1, Location 헤더), /me 200 / (Alice)	작성 201 / 해당 없음 (Location: /api/posts/3), /me 200 / 해당 없음	PASS	

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T06	Alice 토큰으로 본인 글 수정, 삭제	수정 200 / 해당없음, 삭제 204 / 해당없음, 재조회로 반영 확인	수정 200 / 해당 없음, 삭제 204 / 해당 없음, 목록에서 삭제 확인	PASS	  
T07	Bob 토큰으로 Alice 글 수정(PUT) 후 공개 조회	403 / FORBIDDEN, 원문 유지	403 / FORBIDDEN, 재조회 시 원문 유지	PASS	 
T08	Bob 토큰으로 Alice 글 삭제(DELETE) 후 공개 조회	403 / FORBIDDEN, 글 유지	403 / FORBIDDEN, 재조회 시 글 유지	PASS	  

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T09	Alice 토큰의 서명 첫 글자를 바꿔 GET /api/me, POST, PUT, DELETE 요청 후 공개 조회	각각 401 / INVALID_TOKEN, 데이터 변경 없음	401 / INVALID_TOKEN × 4, 재조회 시 글 유지	PASS	  
T10	TTL 3 초로 Alice 로그인, 만료 후 GET /api/me 와 PUT 요청, 화면에서 글 작성	각각 401 / TOKEN_EXPIRED, 화면 로그아웃 상태와 "로그인 시간 만료" 안내	/me 401 / TOKEN_EXPIRED, PUT 401 / TOKEN_EXPIRED, 화면 글 작성 401 / TOKEN_EXPIRED, 로그아웃 상태로 전환	PASS	  
T11	Alice 로그인 → 로그아웃 → 내 정보 조회	헤더에 Authorization 없음, 401 / TOKEN_REQUIRED	401 / TOKEN_REQUIRED, Authorization 헤더 없음	PASS	 
T12	Bob 토큰으로 authorId: 1 추가해 작성, userId: 1 추가해 작성	각각 400 / BAD_INPUT, 위조 글 미생성	400 / BAD_INPUT, 400 / BAD_INPUT, 목록에 위조 글 없음	PASS	  

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T13	Alice 토큰으로 빈 제목, 공백 본문, 101 자 제목을 각각 전송	각각 400 / BAD_INPUT, DB 변경 없음	400 / BAD_INPUT × 3, 재조회 시 글 유지	PASS	
T14	없는 ID(999999) 조회·수정·삭제(정상 토큰), GET /api/posts/abc	없는 글 404 / NOT_FOUND, 잘못된 ID 400 / BAD_INPUT	조회·수정·삭제 404 / NOT_FOUND, abc 400 / BAD_INPUT	PASS	
T15	글 작성 → 백엔드 서버 종료(Ctrl+C) → npm start 로 재실행 → 목록 조회	재조회 200 / 해당 없음, 기존 글의 제목·내용·작성자 유지	재시작 전 200 / 해당 없음, 재시작 후 200 / 해당 없음, 제목·내용·작성자 동일	PASS	
T16	Alice 로그인 후 페이지 새로고침(F5)	목록 200 / 해당 없음, 글 목록 유지, 화면 "로그아웃 상태"	목록 304 / 해당 없음 (브라우저 캐시), 글 목록 유지, 화면 "로그아웃 상태"	PASS	

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T17	토큰 없이 수정·삭제, Bob 으로 글 작성·본인 글 수정·삭제	무토큰 401 / TOKEN_REQUIRED, Bob 작성 201 / 해당 없음, 수정 200 / 해당 없음, 삭제 204 / 해당 없음	무토큰 수정·삭제 401 / TOKEN_REQUIRED, 작성 201 / 해당 없음, 수정 200 / 해당 없음, 삭제 204 / 해당 없음	PASS	    

HTTP 응답 확인 기록

확인 항목	기대 응답	실제 HTTP 와 code	증거 위치
정상 조회·로그인·수정	**200**	200 / 해당 없음	T01, T02, T06
새 글 등록	**201 및 Location**	201 / 해당 없음, Location: /api/posts/3	T05, T17
본인 글 삭제	**204 및 빈 본문**	204 / 해당 없음, 응답 본문 없음	T06, T17
입력 오류·추가 작성자 필드	**400 BAD_INPUT**	400 / BAD_INPUT	T12, T13
잘못된 계정	**401 INVALID_CREDENTIALS**	401 / INVALID_CREDENTIALS	T03
토큰 없음·헤더 형식 오류	**401 TOKEN_REQUIRED**	401 / TOKEN_REQUIRED	T04, T11, T17,
변조·클레임 오류	**401 INVALID_TOKEN**	401 / INVALID_TOKEN	T09
정상 서명의 만료 토큰	**401 TOKEN_EXPIRED**	401 / TOKEN_EXPIRED	T10
다른 작성자의 변경	**403 FORBIDDEN**	403 / FORBIDDEN	T07, T08
없는 글	**404 NOT_FOUND**	404 / NOT_FOUND	T14

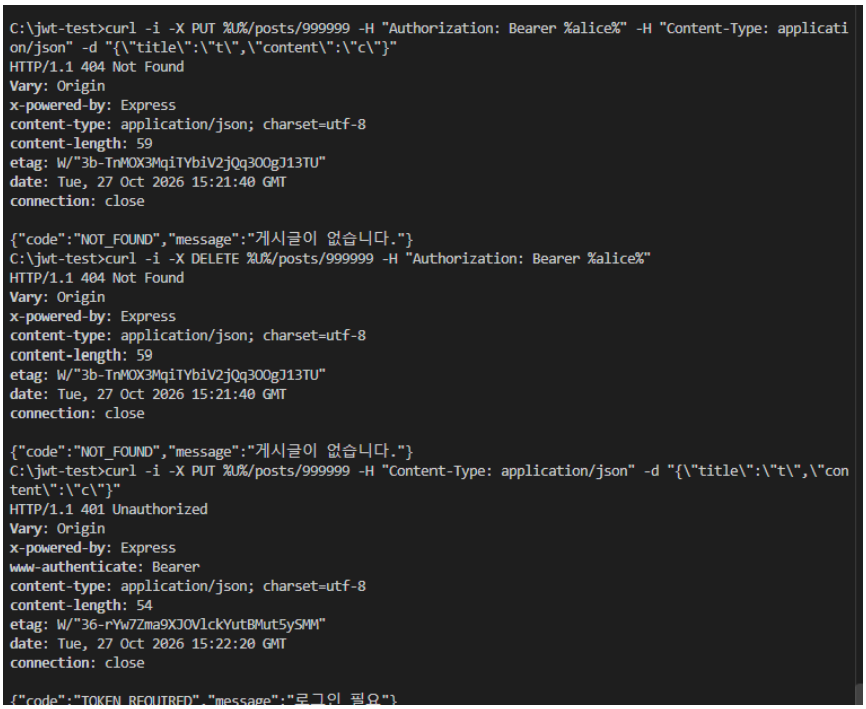
보호 API 의 401 응답에서 응답 헤더에 WWW-Authenticate: Bearer 이 포함된 것을 확인하였다. T04, T09, T10 각각 토큰 없음, 토큰 변조, 토큰 만료의 응답에서 모두 확인하였다. T03 에서 로그인 실패의 401 응답에도 같은 헤더가 포함되었다.

401 응답을 받으면 로그인 상태에서 로그아웃 상태로 변한다. T10 에서 만료된 토큰으로 글을 작성하자 401 TOKEN_EXPIRED 를 반환했고 로그아웃 상태로 바뀌었다.

403 응답은 토큰은 유효하지만 권한이 없는 경우이므로 로그인 상태를 유지하고 문구를 표시한다.

T07,08,09,13 에서 같은 글을 다시 조회하였다. 그 결과 제목, 내용, 작성자 모두 전과 동일하게 유지되는 것을 확인할 수 있었다. 위조 작성자 필드를 넣은 요청 후에도 위조 글이 생성되지 않았다.

존재하지 않는 글을 수정 요청한 결과, 정상 토큰으로 보내면 404 NOT_FOUND, 토큰 없이 보내면 401 TOKEN_REQUIRED 가 반환되었다. 서버가 글 존재 여부보다 인증을 먼저 검사한다.



```
C:\jwt-test>curl -i -X PUT %U%/posts/999999 -H "Authorization: Bearer %alice%" -H "Content-Type: applicati
on/json" -d "{\"title\":\"t\",\"content\":\"c\"}"
HTTP/1.1 404 Not Found
Vary: Origin
x-powered-by: Express
content-type: application/json; charset=utf-8
content-length: 59
etag: W/"3b-TnMOX3MqiTYbiV2jQq3OOgJ13TU"
date: Tue, 27 Oct 2026 15:21:40 GMT
connection: close

{"code":"NOT_FOUND","message":"게시글이 없습니다."}
C:\jwt-test>curl -i -X DELETE %U%/posts/999999 -H "Authorization: Bearer %alice%"
HTTP/1.1 404 Not Found
Vary: Origin
x-powered-by: Express
content-type: application/json; charset=utf-8
content-length: 59
etag: W/"3b-TnMOX3MqiTYbiV2jQq3OOgJ13TU"
date: Tue, 27 Oct 2026 15:21:40 GMT
connection: close

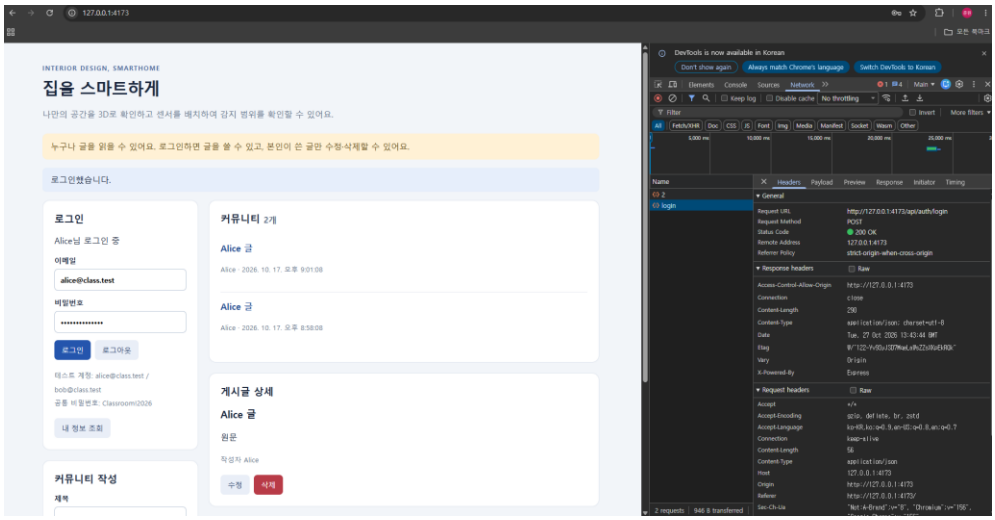
{"code":"NOT_FOUND","message":"게시글이 없습니다."}
C:\jwt-test>curl -i -X PUT %U%/posts/999999 -H "Content-Type: application/json" -d "{\"title\":\"t\",\"con
tent\":\"c\"}"
HTTP/1.1 401 Unauthorized
Vary: Origin
x-powered-by: Express
www-authenticate: Bearer
content-type: application/json; charset=utf-8
content-length: 54
etag: W/"36-rYw7Zma9XJOVlckYutBMut5ySMM"
date: Tue, 27 Oct 2026 15:22:20 GMT
connection: close

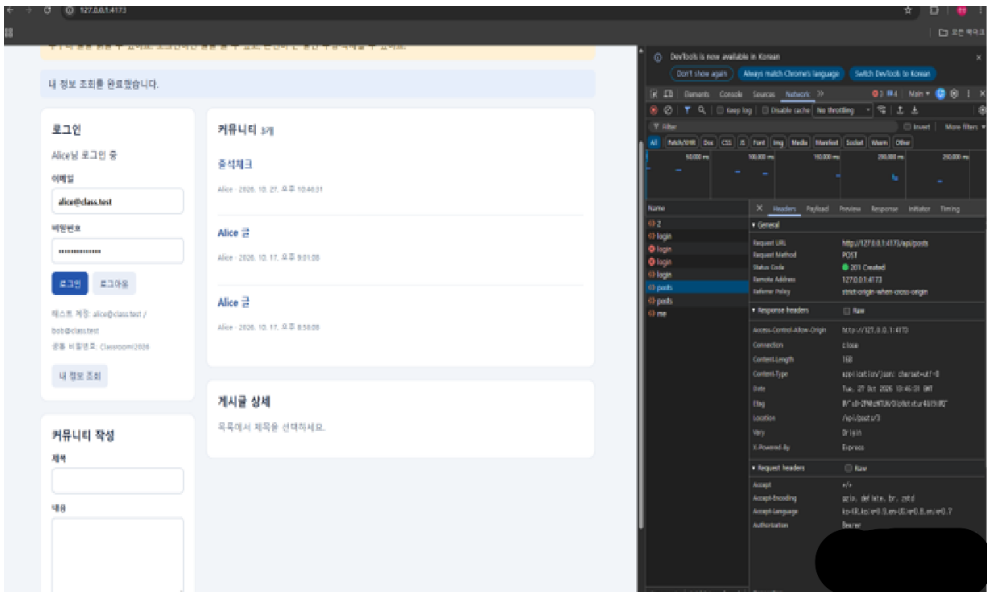
{"code":"TOKEN_REQUIRED","message":"로그인 필요"}
```

테스트 증거

로그인과 토큰 전송

Alice 로 로그인하면 200 OK 와 함께 accessToken 과 사용자 정보를 반환한다. 또한 화면에 로그인 중이 표시되었다. 글 작성 요청에서 토큰이 포함되어 전송되었다.





다른 작성자의 수정과 삭제

Bob 의 정상 토큰으로 Alice 의 글에 수정과 삭제를 요청하였다. 둘 다 403 Forbidden 과 본인이 작성한 글만 수정, 삭제가 가능하다는 말이 반환되었다. 이후 같은 글을 조회한 결과 기존과 같은 글이 유지되었다.

```
C:\jwt-test>curl -i %U%/posts/%P%
HTTP/1.1 200 OK
Vary: Origin
x-powered-by: Express
content-type: application/json; charset=utf-8
content-length: 159
etag: W/"9f-NP30J8b86DIhSSDYTtjVnhGmi4c"
date: Tue, 27 Oct 2026 15:42:17 GMT
connection: close

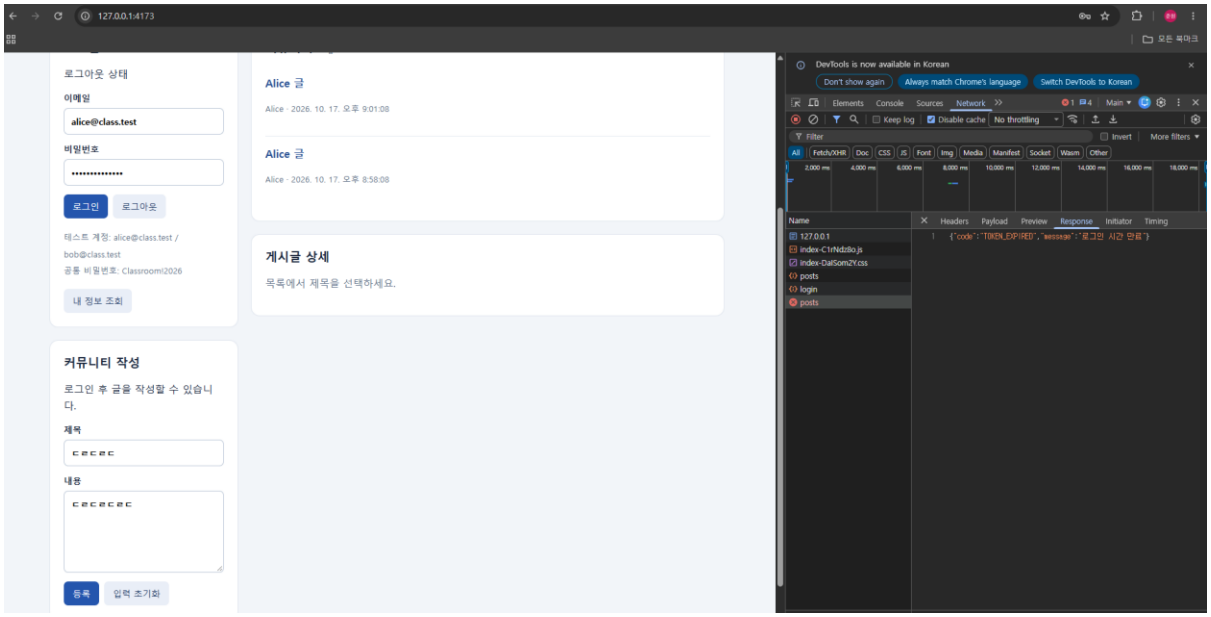
{"id":2,"title":"Alice 글","content":"원문","authorId":1,"authorName":"Alice","createdAt":"2026-10-17T12:0
1:08.819Z","updatedAt":"2026-10-17T12:01:08.819Z"}
```

토큰 변조와 만료

서명 첫 글자를 바꾼 토큰으로 /me, post, put, delete 를 요청하자 모두 401 INVALID_TOKEN 과 헤더가 반환되었다. 이후 글은 전과 바뀌지 않았다. TTL 을 3 초로 설정하여 만료 시간이 지난 요청은 TOKEN_EXPIRED 를 받았다.

```
C:\jwt-test>curl -i %U%/me -H "Authorization: Bearer %alice%"
HTTP/1.1 401 Unauthorized
Vary: Origin
x-powered-by: Express
www-authenticate: Bearer
content-type: application/json; charset=utf-8
content-length: 60
etag: W/"3c-sriZ3eWljQJUxOW+9XSRvmyAmxk"
date: Tue, 27 Oct 2026 15:04:45 GMT
connection: close

{"code":"TOKEN_EXPIRED","message":"로그인 시간 만료"}
 {"code":"INVALID_TOKEN","message":"인증 정보가 올바르지 않음"}
```



자동 검사와 로컬 배포 증거

python tools/check_api.py --mode final 을 실행하여 40 개 항목을 모두 통과하였다. TOKEN_TTL=3 으로 재시작한 서버에서 --expiry 옵션을 붙여 실행했을 때, 48 개 항목을 모두 통과하였다.

프론트엔드에서 빌드에 성공하여 백엔드를 실행한 뒤 http://127.0.0.1:4173 에 접속하였다. 재시작 후에도 같은 글이 조회되었음을 확인할 수 있었다.

```
C:\Users\82107\Documents\web>python tools\check_api.py --mode final
PASS health
PASS public list
PASS bad ID
PASS missing ID
PASS wrong password
PASS unknown email
PASS no token me
PASS no token create
PASS login alice@class.test
PASS login bob@class.test
PASS Alice me
PASS Bob me
PASS create post
PASS public detail
PASS no token PUT
PASS Bob denied PUT
PASS failed request leaves data unchanged
PASS no token DELETE
PASS Bob denied DELETE
PASS failed request leaves data unchanged
PASS reject authorId
PASS reject userId
PASS input validation
PASS input validation
PASS input validation
PASS input validation
PASS tampered GET /me
PASS tampered POST /posts
PASS tampered PUT /posts/7
PASS tampered DELETE /posts/7
PASS failed request leaves data unchanged
PASS owner update
PASS create post
PASS Bob own update
PASS missing protected PUT
PASS missing protected DELETE
PASS delete owned test post
PASS deleted post missing
PASS delete owned test post
PASS deleted post missing
All 40 checks passed (final). UI and restart tests must be performed separately.
```

```
PASS login alice@class.test
PASS login bob@class.test
PASS Alice me
PASS Bob me
PASS create post
PASS public detail
PASS no token PUT
PASS Bob denied PUT
PASS failed request leaves data unchanged
PASS no token DELETE
PASS Bob denied DELETE
PASS failed request leaves data unchanged
PASS reject authorId
PASS reject userId
PASS input validation
PASS input validation
PASS input validation
PASS input validation
PASS tampered GET /me
PASS tampered POST /posts
PASS tampered PUT /posts/9
PASS tampered DELETE /posts/9
PASS failed request leaves data unchanged
PASS owner update
PASS create post
PASS Bob own update
PASS missing protected PUT
PASS missing protected DELETE
PASS login alice@class.test
Waiting 4 seconds for expiration
PASS expired GET /me
PASS expired POST /posts
PASS expired PUT /posts/9
PASS expired DELETE /posts/9
PASS failed request leaves data unchanged
PASS login alice@class.test
PASS login bob@class.test
PASS delete owned test post
PASS deleted post missing
PASS delete owned test post
PASS deleted post missing
All 48 checks passed (final). UI and restart tests must be performed separately.
```

```
npm notice run jwt-board-react@1.0.0 preview
npm notice run vite preview --host 127.0.0.1
  ➜ Local:   http://127.0.0.1:4173/
  ➜ press h + enter to show help
```

```
C:\Users\82107\Documents\web\backend>npm start
npm notice run jwt-board-express@1.0.0 start
npm notice run node src/app.js
SmartHome 커뮤니티 API http://127.0.0.1:8000
```

```
C:\jwt-test>curl -i %U%/posts/%P%
HTTP/1.1 200 OK
Vary: Origin
x-powered-by: Express
content-type: application/json; charset=utf-8
content-length: 159
etag: W/"9f-NP30J8b86DIhSSDYTtjVnhGmi4c"
date: Tue, 27 Oct 2026 15:07:21 GMT
connection: close

{"id":2,"title":"Alice 글","content":"원문","authorId":1,"authorName":"Alice","createdAt":"2026-10-17T12:0
1:08.819Z","updatedAt":"2026-10-17T12:01:08.819Z"}
```

```
C:\jwt-test>curl -i %U%/posts/%P%
HTTP/1.1 200 OK
Vary: Origin
x-powered-by: Express
content-type: application/json; charset=utf-8
content-length: 159
etag: W/"9f-NP30J8b86DIhSSDYTtjVnhGmi4c"
date: Tue, 27 Oct 2026 15:08:35 GMT
connection: close

{"id":2,"title":"Alice 글","content":"원문","authorId":1,"authorName":"Alice","createdAt":"2026-10-17T12:0
1:08.819Z","updatedAt":"2026-10-17T12:01:08.819Z"}
```

AI 활용과 참고 자료

AI 사용 여부: 사용

도구와 모델	사용 목적	반영한 파일 또는 보고서 항목	반영 방식
Claude Code / Claude Opus 5.5	학습	JavaScript 문법, JWT 구조, HTTP 상태 코드 설명	참고
Claude Code / Claude Opus 5.5	디버깅	backend/src/auth.js, frontend/src/api.js, frontend/src/main.jsx (본인 작성 코드 검토)	수정 후 반영 (AI 가 오류를 지적하고 본인이 수정)
Claude Code / Claude Opus 5.5	생성	auth.js `currentUserId` 의 `jwt.verify` 옵션, api.js `login` 의 요청 줄	그대로 반영
Claude Code / Claude Opus 5.5	생성	프로젝트 폴더 구성, 의존성 설치, `.env` 생성, 코드 정리(비교 연산자, 공백, 서버 시작 메시지)	그대로 반영 (AI 가 수행)
Claude Code / Claude Opus 5.5	테스트	curl 테스트 명령, 토큰 저장용 `login.bat` (저장소에 포함하지 않음)	참고
Claude Code / Claude Opus 5.5	문장정리	보고서 1~5 장	수정 후 반영

대표 사례

도구 / 날짜: Claude Code (Claude Opus 5.5) / 2026-10-20

요청한 내용: BE-02(currentUserId)를 작성한 뒤 "B 다 썼어 봐줘"로 검토를 요청하였다

AI 가 제안한 내용: jwt.verify 의 옵션에 jwt.sign 용 옵션(subject: String(user.id) 등)을 그대로 넣어, 함수 안에 없는 user 를 참조하고 있다고 지적하였다. 이 오류가 catch 에 잡혀 모든 토큰이 INVALID_TOKEN 으로 처리된다는 점도 설명하였다. 그리고 검증용 옵션 {algorithms: ["HS256"], issuer: ISSUER, audience: AUDIENCE}를 제시하였다.

실제 반영 위치: backend/src/auth.js currentUserId

- 내가 확인하고 수정한 내용: 제시된 옵션을 그대로 반영하였다.
- 같은 검토에서 다음 오류도 지적받아 직접 수정하였다.
    - 서명 키로 원본 문자열 key 대신 변환한 new_key 를 써야 함
    - 옵션 이름은 algorithm 이 아니라 algorithms 임
    - findPublic import 누락
    - id <= 0 검사로는 NaN 이 통과함 → Number.isInteger 로 변경

- exp 가 없는 토큰이 통과함 → typeof payload.exp 검사 추가

검증 방법과 결과:

- 정상 토큰으로 /me 를 요청하면 200, 서명을 바꾼 토큰은 401 INVALID_TOKEN(T09), 만료 토큰은 401 TOKEN_EXPIRED(T10)가 반환되었다.
- 수정 전에는 정상 토큰도 401 INVALID_TOKEN 이었다.

도구 / 날짜: Claude Code (Claude Opus 5.5) / 2026-10-20

요청한 내용: "alice 로 글 쓰기 계속 로그인시간만료 문제

AI 가 제안한 내용 코드 문제가 아니라 실행 환경 문제라고 진단하였다. 만료 테스트를 위해 cmd 에서 실행한 set TOKEN_TTL=3 이 그 터미널을 닫을 때까지 유지되어, 이후 실행한 서버도 3 초짜리 토큰을 발급하고 있었다. 해결 방법으로 터미널을 닫고 새로 열어 서버를 다시 실행하라고 안내하였다.

실제 반영 위치: 코드 변경 없음 (실행 절차)

내가 확인하고 수정한 내용: 새 터미널에서 서버를 다시 실행하여 해결하였다.

검증 방법과 결과:

새 터미널에서 서버를 실행한 뒤 로그인 응답의 expiresIn 값을 확인하였고, 글 작성이 201 로 성공하였다.

참고 자료

자료 제목과 URL	사용한 위치	반영 내용 및 출처 표시
웹서비스설계 과제 공지, 보고서 양식, 테스트 시나리오, 제공 템플릿 (수업 자료)	프로젝트 전체, 보고서, 4 장 테스트	제공 코드(CRUD, SQLite 저장, 입력 검사, 오류 응답 형식) 유지, T01~T17 절차

이해도 확인과 회고

1. 요청 본문의 authorId 를 믿으면 어떤 문제가 생기며, 내 코드는 어떻게 막는가?

요청 본문은 클라이언트가 자기 마음대로 바꿀 수 있다. 서버가 본문의 authorId 를 그대로 저장하면 Bob 이 Alice 이름으로 글을 쓸 수 있다. 그러면 그 글은 DB 상 Alice 의 글이 되어 Bob 이 수정, 삭제도 가능해진다.

App.js 에서 글 작성 라우트는 작성자를 본문이 아니랏 currentUserId 가 반환한 값으로 정한다. 이 값은 서명이 검증된 토큰의 sub 에서 나온다.

App.js 에서 fields()함수에서 본문에 title 과 content 외의 필드가 있으면 400 BAD_INPUT 으로 거부한다.

T12 에서 Bob 이 authorId:1, userId : 1 을 각각 넣어 보낸 요청 모두 BAD_INPUT 이 반환되었다.

2. 수정 버튼을 숨기는 것만으로 충분하지 않은 이유는 무엇이며, 서버의 어떤 함수가 권한을 검사하는가?

버튼 숨기기는 브라우저 화면에서 일어난다. API 는 화면이 필요하지 않고 curl 로 직접 호출이 가능하다. PUT /api/posts/2 를 보내는 것을 막지 못한다.

실제 권한 검사는 서버의 assertOwner 가 한다. 이 함수는 토큰에서 검증한 요청자 ID 와 DB 에 저장된 글의 authorId 를 비교하고 다르면 403 Forbidden 을 준다. 수정과 삭제는 DB 전에 발생한다. T07, 08 을 보면 403 이 반환되었고 글은 남아 있었다.

3. JWT decode 와 verify 의 차이는 무엇이며, 내 코드는 어떤 검증을 수행하는가?

Decode 는 토큰의 payload 를 Base64URL 로 풀어서 읽기만 하고 서명은 확인하지 않는다. JWT 의 내용은 암호화 된 것이 아니라 누구나 읽고 고칠 수 있다. Decode 만 쓰면 sub 를 바꾼 위조 토큰으로도 속일 수 있다.

Verify 는 서버의 비밀 키로 서명을 다시 해 토큰의 서명과 비교하므로 내용이 다르면 거부한다.

Authorization 헤더가 Bearer 로 시작하는지 확인한다. 아닐 시 401 TOKEN_REQUIRED 를 반환한다.

Jwt.verify 로 서명, 알고리즘, iss, aud, 만료를 검사한다.

Exp 가 정수인지 확인한다.

Sub 를 정수로 바꿔 확인한다.

실제 존재하는 사용자인지 확인한다.

4. 로그아웃, 토큰 만료, 새로고침은 각각 무엇을 바꾸는가? 이미 복사된 토큰은 어떻게 되는가?

로그아웃은 브라우저 메모리의 토큰과 사용자 정보를 지운다. 서버는 아무것도 몰라 토큰 자체는 바뀌지 않는다.

토큰 만료는 exp 시각이 지나면 서버의 verify 가 토큰을 거부한다. 브라우저는 세션을 지우고 로그아웃 상태로 바뀐다. 토큰 문자열은 바뀌지 않는다.

새로고침은 토큰을 메모리 변수에만 저장하므로 새로고침을 하면 사라진다. 화면은 로그아웃 상태가 된다. 글 목록과 DB 는 바뀌지 않는다.

복사된 토큰은 로그아웃, 새로고침과 관계없이 유효하다. 서버는 토큰 목록을 저장하지 않고 서명과 만료만 검사한다.

5. 가장 어려웠던 오류 한 가지와 원인 확인·수정·재검증 과정을 설명하라.

BE-02 에서 모두 401 INVALID_TOKEN 으로 거부되었을 때였다.

원인으로 jwt.verify 의 옵션이 jwt.sign 에 쓰던 옵션을 그대로 복사하고 있었고 currentUserId 안에는 user 변수가 없었다. 오류가 catch 에 잡히면서 토큰 오류로 바뀌어 몰랐고 발급과 검증의 옵션을 알게 되었다.

verify 옵션을 {algorithms: ["HS256"], issuer: ISSUER, audience: AUDIENCE}로 바꾸었다. key 대신 32 바이트 키를 쓰도록 고쳤고 id <= 0 검사는 NaN 을 통과시키므로 Number.isInteger 로 바꾸었다.

정상 토큰의 /me 가 200 으로 바뀐 것을 확인하였다. 서명 변조 토큰(401 INVALID_TOKEN), 만료 토큰(401 TOKEN_EXPIRED), 토큰 없음(401 TOKEN_REQUIRED)이 동작하는 지 확인하였다.

현재 남은 문제와 다음에 개선할 점: 토큰을 메모리에만 저장하므로 새로고침 시 다시 로그인해야하는 번거로움이 존재한다.